

EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks

Mingxing Tan & Quoc V. Le (Google Research)

Deaconu Mihai Bogdan

March 19, 2026

Core Concepts & Terminology

- **ConvNet:** Neural network structured for grid data (images), built with composed stages of shared layer architectures.
- **Backbone:** The primary feature extractor network. We attach varying *Necks* (for aggregation) and *Heads* (for specific tasks like bounding boxes) to it. EfficientNet serves as a backbone.
- **FLOPS:** Floating Point Operations. A hardware-agnostic measure of computational complexity.
- **NAS (Neural Architecture Search):** Using algorithms to automatically search a vast design space to find the optimal neural network structure.
- **Skip Connections:** Structural shortcuts that bypass layers, helping gradients flow smoothly during backpropagation to prevent the vanishing gradient problem.

The Scaling Problem

The Standard Deep Learning Workflow:

- Engineers develop ConvNets for a fixed computational resource budget.
- When better hardware is available, models are scaled up for better accuracy.

Examples of Arbitrary Scaling:

- *ResNet*: Scaled from ResNet-18 to ResNet-200 by adding more layers.
- *GPipe*: Scaled up to an enormous 557M parameters using massive resolution (480×480) and complex pipeline parallelism.

The Issue: Scaling arbitrary dimensions requires tedious manual tuning and invariably yields sub-optimal parameter efficiency.

Single Dimension Scaling: Pros & Cons

- **Depth (d):** Adding more layers.
 - *Pros:* Captures richer, highly complex features.
 - *Cons:* Vanishing gradients. Even with skip connections, the accuracy gain rapidly diminishes (e.g., ResNet-101 $\approx$ ResNet-1000).
- **Width (w):** Adding more channels.
 - *Pros:* Captures fine-grained features; easier to train.
 - *Cons:* Extremely wide, shallow networks struggle to capture high-level abstract features.
- **Resolution (r):** Feeding larger input images.
 - *Pros:* Better fine-grained pattern recognition.
 - *Cons:* Extreme FLOPS and memory overhead; accuracy gain diminishes at very high resolutions.

Observation 1

- Scaling up any single dimension (width, depth, or resolution) improves accuracy, but the accuracy gain quickly saturates for bigger models.

Observation 2: The Breakthrough

- To pursue maximal accuracy and efficiency, it is critical to **balance all dimensions** simultaneously.
- *Intuition:* A higher resolution image provides a larger spatial area with more pixels. Therefore, the network needs to be **deeper** to increase the receptive field, and **wider** to capture the additional fine-grained pixel patterns.

Visualizing Different Scaling Methods

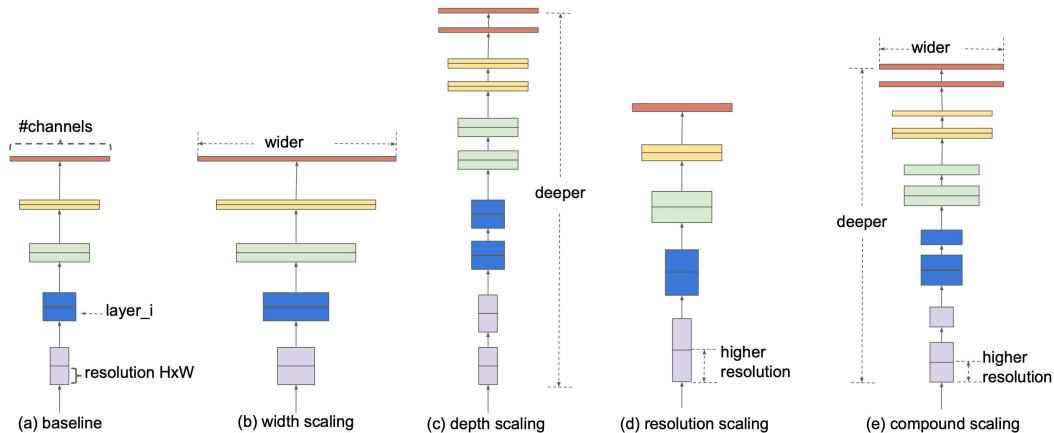


Figure 2: Comparing single-dimension scaling vs. Compound Scaling

The Compound Scaling Method

The authors introduced a compound coefficient (ϕ) to uniformly scale all dimensions:

$$d = \alpha^\phi \quad w = \beta^\phi \quad r = \gamma^\phi$$

The Constraint: $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$

Why this exact formula?

- The FLOPS of a standard convolution scale proportionally to depth (d), width squared (w^2), and resolution squared (r^2).
- By setting the product of these constants to ≈ 2 , increasing the coefficient ϕ by 1 mathematically guarantees that the total FLOPS of the network exactly **double** (2^ϕ).

Is this approach Model-Agnostic?

Yes. The researchers applied compound scaling to standard existing baselines:

- **MobileNetV1 & V2:** Compound scaling vastly outperformed arbitrarily scaling width or resolution alone.
- **ResNet-50:** Applying ϕ to balance ResNet-50 resulted in better top-1 accuracy than pushing it to ResNet-100+ via depth alone.

However, scaling a sub-optimal baseline still yields a sub-optimal large model. A new, mathematically optimal baseline was required.

Developing the Baseline: EfficientNet-B0

- The authors used **Multi-objective NAS** optimizing for both Accuracy and FLOPS.
- The resulting architecture is **EfficientNet-B0**.
- Its core component is the **MBConv Layer**.

Stage i	Operator $\hat{\mathcal{F}}_i$	Resolution $\hat{H}_i \times \hat{W}_i$	#Channels $\hat{C}_i$	#Layers $\hat{L}_i$
1	Conv3x3	224×224	32	1
2	MBConv1, k3x3	112×112	16	1
3	MBConv6, k3x3	112×112	24	2
4	MBConv6, k5x5	56×56	40	2
5	MBConv6, k3x3	28×28	80	3
6	MBConv6, k5x5	14×14	112	3
7	MBConv6, k5x5	14×14	192	4
8	MBConv6, k3x3	7×7	320	1
9	Conv1x1 & Pooling & FC	7×7	1280	1

Table 1: EfficientNet-B0 Architecture

EfficientNet derives its efficiency from three innovations within the MBConv layer:

- **Depthwise Separable Convolutions:** Splits a standard convolution into a spatial step (depthwise) and a channel-mixing step (pointwise), drastically cutting parameters and FLOPS.
- **Inverted Residuals:** Instead of compressing features (bottleneck), it expands the channels, applies the cheap depthwise convolution, and then compresses them back down, connecting the narrow layers with a skip connection.
- **Squeeze-and-Excitation (SE):** A lightweight attention mechanism that pools spatial dimensions to judge channel importance, dynamically amplifying useful features and suppressing noise.

Generating the EfficientNet Family (B0 to B7)

Step 1: Grid Search on the B0 Baseline

- Set $\phi = 1$ (assuming exactly twice the computational resources).
- A small grid search found the optimal constants: $\alpha = 1.2$, $\beta = 1.1$, $\gamma = 1.15$.

Step 2: Scale up via ϕ

- Lock the constants α, β, γ .
- Increment ϕ to continuously generate larger, uniformly scaled models: EfficientNet-B1 through B7.

State-of-the-Art Results: Full Progression

Model	Top-1 Acc.	Params	Ratio (Params)	FLOPS	Ratio (FLOPS)
EfficientNet-B0	77.1%	5.3M	1x	0.39B	1x
ResNet-50	76.0%	26M	4.9x	4.1B	11x
EfficientNet-B1	79.1%	7.8M	1x	0.70B	1x
ResNet-152	77.8%	60M	7.6x	11B	16x
EfficientNet-B2	80.1%	9.2M	1x	1.0B	1x
EfficientNet-B3	81.6%	12M	1x	1.8B	1x
EfficientNet-B4	82.9%	19M	1x	4.2B	1x
SENet	82.7%	146M	7.7x	42B	10x
EfficientNet-B5	83.6%	30M	1x	9.9B	1x
EfficientNet-B6	84.0%	43M	1x	19B	1x
EfficientNet-B7	84.3%	66M	1x	37B	1x
GPipe	84.3%	557M	8.4x		

Conclusion & Where it Stands Today

- **Is it still used?** Absolutely. It remains a gold standard benchmark for parameter efficiency, heavily utilized on edge and mobile devices.
- **Why is ResNet still popular?** Hardware optimization. Modern GPUs (via CuDNN) are incredibly optimized for standard 3×3 convolutions. Depthwise separable convolutions (MBConv) are mathematically cheaper but can be memory-bandwidth bound, meaning ResNets sometimes train faster in wall-clock time despite having more parameters.
- **Key Takeaway:** Mathematical, balanced compound scaling unlocks an architectural efficiency that arbitrary manual tuning can never achieve.

Questions?